

Contract-Driven Harness Engineering for Reliable Low-Cost Agent Tasks

Contract-Driven Harness Study

June 15, 2026

Language-edited Version 4 derivative of the frozen v3.1.1 body. External literature citations remain as BibTeX keys, and Appendix C and the reproducibility package preserve the empirical evidence trail.

Abstract

When a productivity agent drops evidence, loses state, skips a stage gate, or omits a required field, the failure is often attributed to the model. For bounded tasks, the immediate problem may instead be that these obligations were never represented in an inspectable form. Contract-driven harness engineering represents task obligations through task specifications, bounded memory slices, evidence bundles, and output contracts. Workflow gates and validators enforce those obligations, while trace requirements record whether they were preserved.

The question is deliberately narrow. We do not test whether a harness makes a low-cost model generally equivalent to a strong model. We ask whether explicit obligations make failures in low-cost-model runs easier to inspect, repair, and cover with regression tests.

Experiments cover structured extraction, project initialization, research workflow, mechanism atoms, admitted macros, and controlled state mutation. Harnessing raises absolute contract adherence across these settings. It also compresses model gaps when the baseline gap is nonzero and the task is highly constrained, but that result is not universal. The more consistent finding is weak-model enablement on bounded, contract-critical operations.

The method starts with mechanisms rather than whole workflows. Broad tasks are decomposed into testable mechanisms. Admission requires golden, known-bad, and local-gate checks, and macro scope expands only when carried obligations are explicit. In a fresh stability confirmation, Qwen3-8B under a frozen explicit-transition-delta G9 protocol passed 40/40 controlled state-mutation runs across five perturbation conditions (95% Wilson interval: [0.912, 1.000]). A preceding paired ablation favored the explicit-delta arm but did not meet the preregistered engineering-effect threshold. These results support bounded protocol stability and weak-model enablement. They do not establish a large independent causal effect, production readiness, or open-ended workflow reliability.

1 Introduction

Agent systems perform routine productivity work such as project initialization, structured extraction, evidence synthesis, plan preparation, document updates, and multi-step coordination. Failures in these settings are often read as direct evidence of insufficient model capability. When an agent loses a constraint, omits evidence, skips a stage, or reuses stale context, the model is the obvious suspect.

Model capability can be the cause, but it is not the only one. Many obligations are known before generation begins: admissible evidence, known and unknown state, blocked actions, required fields,

citation rules, and the stage gate that prevents a premature recommendation. When these obligations remain implicit, the model must recover and retain them while it generates. Representing them as contracts moves part of that burden into the surrounding system.

We use the term contract-driven harness engineering for this system layer. It represents task obligations through specifications, bounded memory, evidence bundles, and output contracts. Workflow gates and validators enforce those obligations, while trace requirements record their treatment during execution. The research question does not assume that low-cost models are equivalent to strong models. It asks which reliability requirements become less dependent on unconstrained generation once task obligations are inspectable.

Three sources of failure need to be kept separate. Model capability concerns reasoning, instruction following, and recovery from ambiguity. Harness specification concerns whether task obligations, admissible evidence, known and unknown state, output structure, and blocked actions are stated explicitly. Workflow composition concerns whether those obligations are preserved across steps, tool calls, and state transitions. The evidence in this paper covers harness specification and bounded composition, not open-ended workflow autonomy.

The project initially asked whether a stable harness could compress the measured gap between strong and low-cost models. The answer depended on the task. Under G9, measured nonzero baseline gaps compressed in highly structured extraction. In the broader project-initialization and research-workflow slices, G9 improved absolute contract adherence while gap movement remained mixed or undefined. Those slices also exposed a measurement problem: one workflow-level score can conceal failures in schema following, state retention, evidence grounding, stage discipline, or trace completeness.

This problem led to mechanism-first evaluation. A mechanism atom is a fixed-input, deterministic operation bound by an explicit contract. Each atom isolates one primary mechanism and one dominant failure mode, and includes a golden output, a known-bad output, and a composition interface. The unit makes narrower questions possible. Evidence bundles can be tested for claim grounding, and memory slices for whether they prevent state hallucination. Stage gates can be tested against premature recommendation, while trace requirements can be evaluated through the auditability of rejection paths. Atom-level success does not establish workflow-level reliability, but it makes later composition failures easier to locate.

The repair loop provided the study’s most complete failure-isolation and repair sequence. Stage 7e composed state inventory, evidence grounding, evidence typing, traceable decision, and stage-gated synthesis into a narrow evidence-bound macro. Its first version showed that the low-cost model could pass under one harness arm yet lose decision-trace and stage-gate obligations under another. Stage 7e v2 retained the decision trace and stage gate, but some outputs still omitted unknown state. Stage 7e v3 added an explicit unknown-state requirement. Those omissions disappeared, although one output still reduced known-state provenance to generic labels. Stage 7e v4 made that provenance explicit and passed 4/4 targeted smoke runs after retrying a provider timeout and a truncated output.

The Neighboring Macro Transfer study (Stage 7-next) tested whether the repair extended beyond the original fixture. It reused the Stage 7e v4 obligations in a neighboring evidence-bound method-plan update macro. The output contract required the model to identify the next admitted macro, list its admission criteria, preserve the local and real-model gates, and declare non-claims. Qwen3-8B passed 4/4 targeted smoke runs under G8/G9; every run scored 1.000 on task success and the strict primary macro metric. The scope is narrow: the result supports transfer to one closely related bounded macro, not to broader workflow classes.

The Controlled State-Mutation Study (Stage B) subjected a repaired state-mutation obligation to stricter evaluation. Stage B v5 exposed failures in evidence arrays and the gate. The v5.1 revision

repaired the complete gate and immutable evidence bindings. A preregistered v5.2 ablation did not find an engineering-scale independent effect from evidence-binding separation. Stage B v5.3 then isolated an explicit transition delta: the delta arm passed 15/15, compared with 13/15 for the exact-postcondition baseline. The direction favored the delta arm, but the difference remained below the preregistered effect threshold. Stage B v5.4 addressed a separate absolute question using only the frozen delta protocol. It passed 40/40 fresh runs across canonical, field-alias, evidence-order, distractor-evidence, and unknown-state-paraphrase conditions, with zero provider errors and zero retries.

These results narrow the original gap-compression thesis. A contract-driven harness neither makes weak models generally equivalent to strong models nor guarantees gap compression. It can raise the usable floor of a low-cost model on bounded, contract-critical operations. It can also turn some failures into repairable objects. A missing obligation can be named and added to the contract, then captured in a known-bad case and checked locally. The revised contract can be rerun against the model and recorded in the evidence ledger and claim boundary.

1.1 Contributions

The paper makes five contributions:

1. We define contract-driven harness engineering as an explicit reliability layer for agent tasks.
2. We propose mechanism atoms as the unit of harness evaluation.
3. We report a multi-stage empirical evaluation across task slices, mechanism atoms, and admitted macros.
4. We introduce a repair-loop protocol for harness development.
5. We provide bounded evidence that Qwen3-8B maintained strict contract adherence on one frozen controlled-state-mutation protocol across 40 fresh runs and five designed perturbation conditions.

2 Related Work

Relevant prior work spans agent orchestration, declarative LM programming, structured output constraints, retrieval and tool augmentation, memory systems, safety verification, and skill ecosystems. Across these areas, obligations that might otherwise remain inside free-form generation are moved into runtime, specification, tool, memory, validation, or evaluation layers.

The distinction is summarized in Table 1.

Work family	Main focus	What it externalizes	What this paper adds
Workflow orchestration	execution graph and state	steps, tools, persistence, human checkpoints	obligation-level evaluation and repair
Structured outputs	syntactic output control	schema and format constraints	semantic contract obligations such as evidence, unknown state, and blocked claims

Work family	Main focus	What it externalizes	What this paper adds
Guardrails and validators	runtime checks and retries	validation policies and failure handling	known-bad-driven repair loops tied to mechanism atoms
Declarative LM programs	modules, signatures, metrics	program structure and optimization targets	mechanism-first empirical repair for low-cost-model enablement
Agent specifications	portability and interface contracts	workflow, state, and step definitions	evidence-bound admission criteria before macro composition
Retrieval, tools, and memory	external knowledge and actions	documents, APIs, tool calls, long-term state	bounded memory/evidence contracts before live side effects
Safety and verification	policy compliance and assurance	constraints, static checks, runtime firewalls	empirical contract adherence with explicit non-claims

2.1 Agent Workflows And Orchestration

Recent agent engineering guidance distinguishes autonomous agents from workflows with explicit control paths. Anthropic’s discussion of effective agents places predictable, decomposable tasks in the workflow category and reserves agents for cases that require open-ended model autonomy. LangGraph, AutoGen, Semantic Kernel, and related orchestration frameworks make execution state, graph structure, persistence, human intervention, tool calls, and observability properties of the system rather than of the prompt alone. [2, 8, 12, 13]

These systems make execution durable and inspectable, and they provide integration points for tools and people. A workflow graph can still route every step correctly while losing evidence provenance, collapsing unknown state, skipping a stage gate, or producing an ungrounded recommendation. Orchestration alone therefore does not settle the evaluation problem addressed here.

Contract-driven harness engineering overlaps with orchestration, but the graph is only one layer. The evaluation follows state and evidence obligations, including state inventory, evidence binding, and evidence type separation. It also checks whether trace, stage-gate, and excluded-context obligations remain auditable and repairable across the graph.

2.2 Declarative LM Programs And Agent Specifications

Declarative LM programming systems, especially DSPy, argue that language model behavior should be represented as programs with signatures, modules, metrics, and optimizers rather than as hand-written prompts. Agent specification work such as AgentSPEX and AgentSpec pushes in a related direction for agent systems: workflows, state, steps, and interfaces should be declared in portable and inspectable forms. [7, 19, 1]

This line of work is close to the method used here because both make task structure explicit. The evaluation target differs. Declarative systems often emphasize program optimization, portability, or agent specification. Here, each contract-critical obligation is defined and paired with golden and known-bad outputs. Deterministic local gates and a small real-model slice are run before the claim boundary is updated and a broader workflow is composed.

2.3 Structured Outputs, Guardrails, And Validators

Structured output systems and guardrail frameworks externalize output form. OpenAI structured output mechanisms, Outlines-style constrained generation, and Guardrails validators reduce the burden of asking a model to follow a format. They make schema adherence and selected validation checks part of the system layer. [14, 5, 6]

These mechanisms address part of the problem. Schema validity can ensure that an output has the expected shape, but it does not establish that a claim is supported, unknown state remains unknown, excluded context is not reused, or a recommendation is blocked when a stage gate is incomplete. Validators are therefore one component of the contract stack. In addition to fields, the output contract specifies evidence IDs, evidence type separation, rejected-option traces, blocked outputs, and non-claims.

2.4 Retrieval, Tools, Memory, And Externalized Capability

RAG, ReAct, Toolformer, Gorilla, and tool/API-focused agent work show that models can become more capable when knowledge and action are externalized. Retrieval can provide updated evidence and provenance. Tool-use frameworks can turn external actions into typed calls. API benchmarks show that tool descriptions and retrieval can materially improve call generation compared with unaided model behavior. [11, 21, 17, 16]

Memory-oriented systems such as MemGPT and Letta show that agents can use hierarchical, archival, or stateful memory to extend beyond a single context window. They also expose a reliability problem for this study: memory is not automatically beneficial. A system must decide what to store and how narrowly to scope it. It must also decide when to summarize or retrieve state, and how to prevent stale or irrelevant context from contaminating a new task. [15, 10]

Live retrieval, live tool execution, and long-term memory are outside the primary evaluation. Most admitted mechanism atoms and macros use fixed inputs and no tools. This restriction isolates contract adherence before changing corpora, live tools, or runtime side effects are introduced.

2.5 Evaluation, Safety, Verification, And Skill Ecosystems

Agent evaluation and safety work highlights the fragility of agent claims. OAgents-style critiques emphasize protocol variance and reproducibility challenges. Semantic Integrity Constraints, Agent-proof, LlamaFirewall, and related verification or guardrail systems argue that agent behavior must be constrained, audited, or checked against explicit policies and semantic rules. [22, 9, 20, 3]

Capability ecosystems such as MCP servers, agent skills, registries, pack systems, and PEFiSh-style skill markets provide another route to harness engineering. They externalize reusable procedures and tool access, while also tracking installation state, platform routing, quality gates, and capability discovery. PEFiSh supplies the experimental setting. Its packs, skills, and MCP servers provide reusable capabilities; installers, trigger evaluators, quality gates, and context plugins govern how those capabilities are selected and checked. Appendix C and the reproducibility package preserve the local PEFiSh-specific evidence.

PEFiSh is the implementation context, not the transferable claim. The contract stack represents obligations through task specs, bounded memory, evidence bundles, and output contracts. Workflow gates, trace logs, validators, known-bad cases, and claim-boundary updates support enforcement, diagnosis, and repair.

The remaining gap is mechanism-level evaluation: which explicit obligations let a low-cost model complete bounded, contract-critical operations, and how should the harness change when an obligation fails?

Declarative LM programs, guardrail or validator systems, and agent specification languages are the closest prior lines. They externalize structure, interfaces, or runtime checks. Our procedure treats each missing reliability obligation as an empirical repair target, requires golden and known-bad fixtures before admission, and restricts macro claims to obligations that survive composition. The unit of evaluation is the obligation that remains auditable across the harness, rather than the workflow graph or schema alone.

3 Methods

3.1 Study Design

The evaluation asks whether explicit harness contracts make productivity tasks less dependent on unconstrained model behavior. A complete agent workflow is not the primitive benchmark unit. Harness behavior is examined at three levels:

1. task slices, which compare broad task classes across harness strengths;
2. mechanism atoms, which isolate a single primary harness mechanism and a dominant failure mode;
3. admitted macros, which compose only mechanisms that have passed local gates and targeted model checks.

Broad workflow results motivate failure analysis, but they do not by themselves justify a general harness claim. A workflow enters the main claim only after its component mechanisms, local evaluators, known-bad cases, and cross-step obligations are explicit.

The experimental path therefore moves from broad failures toward admitted composition rather than from one benchmark score to a larger benchmark score:

Task slices

```
| - structured extraction
| - project initialization
' - research workflow
    -> failure analysis
```

Mechanism atoms

```
| - Stage 6
| - Stage 7r
' - Stage 7r.1
    -> admission gate
```

Admitted macros

```
| - Stage 7p / 7p v2
| - Stage 7e v1-v4
| - Stage 7-next
' - Stage B v5-v5.4
```

The map captures the experimental sequence. Broad tasks expose unstable behavior; mechanism atoms isolate the obligation; local gates reject known-bad outputs; and an admitted macro carries the obligation forward.

Internal stage ID	Paper-facing label	Purpose
Stage 6	Mechanism-Atom Pilot	Test isolated harness mechanisms.
Stage 7p	Partial Macro Composition	Test whether passing atoms compose.
Stage 7r / 7r.1	Atom Revision and Repair	Repair boundary-prone atoms.
Stage 7e	Evidence-Decision Macro Repair	Apply the iterative macro repair loop.
Stage 7-next	Neighboring Macro Transfer	Test narrow obligation transfer.
Stage B	Controlled State-Mutation Study	Separate repair, ablation, and stability evidence.

The stage identifiers are retained from the reproducibility package for traceability; descriptive labels are used in the paper to clarify each stage’s role.

3.2 Harness Model

A contract-driven harness consists of explicit control objects around a language model:

Object	Role
TaskSpec	Objective, constraints, success conditions, and non-goals.
MemorySlice	Bounded context that may be used, plus excluded or unknown state.
EvidenceBundle	Admissible evidence items, evidence types, and source links.
OutputContract	Required output shape, nested fields, citation policy, and validator rules.
WorkflowGraph or stage gate	Required order of intermediate steps and blocked outputs.
TraceLog	Decision trace requirements for auditable reasoning and rejection paths.
ValidatorGate	Deterministic local checks that distinguish passing outputs from known-bad outputs.

The working assumption is that, for a bounded task, reliability requirements should move from implicit model judgment into explicit, inspectable contracts where possible. The harness is evaluated as a reliability-engineering layer. No equivalence between low-cost and strong models is assumed.

The object of study is the lifecycle of an obligation: from an observed omission to a contract field, a known-bad fixture, a deterministic gate, and an admitted macro requirement.

3.3 Harness Arms And Models

Harness arms vary the amount of external control. G0 provides raw or minimally constrained task input. G2/G3 are intermediate mechanism arms where applicable. G8 adds contract-rich execution with validator or evaluator obligations. G9 supplies the full packet: task specification and output contract, evidence and memory policy, plus workflow, trace, and regression expectations.

Real-model slices use SiliconFlow’s OpenAI-compatible API. The current low-cost tier is `Qwen/Qwen3-8B`; earlier strong-model slices used `deepseek-ai/DeepSeek-V3.2`. Provider-backed runs use temperature 0 and prompts exported before execution. Each run has its own artifact directory

containing the adapter request, output, validation report, metrics, and event logs for provider start and end, elapsed time, errors, and retries. [18]

3.4 Task Slices, Mechanism Atoms, And Macros

Broad task slices form the first empirical layer. They show where harnessing helps and where a workflow-level definition becomes too noisy. Structured extraction is a high-constraint task with deterministic output structure. Project initialization adds multiple workspace-planning constraints. Research workflow evaluates evidence-backed synthesis.

A mechanism atom is the smallest testable unit of harness behavior. It isolates one primary mechanism on a fixed input under an explicit output contract. Each atom also has a deterministic evaluator, a known-bad rejection case, a pass threshold, and a composition interface. Admission requires a valid fixture and a passing golden output, together with at least one known-bad output that fails for the intended reason. The baseline must leave room for improvement, or the evaluation must explicitly target absolute adherence; the low-cost model must then improve or reach the pass threshold under the relevant arm. The downstream composition interface must also be declared.

Macro composition begins only after the component mechanisms pass local gates and targeted model checks. Cross-step obligations must be carried explicitly. The admitted macro family remains fixed-input: Stage 7p v2, Stage 7e v1-v4, Stage 7-next, and the Stage B controlled-state-mutation sequence. Project initialization and full research workflow remain blocked because the evidence covers bounded macros rather than open-ended, tool-using workflows.

3.5 Admission Criteria

An atom or macro is admitted to the next experimental layer only if all of the following hold:

1. the fixture schema validates;
2. the golden output passes;
3. at least one known-bad output fails for the intended reason;
4. the baseline leaves improvement room or the evaluation question explicitly targets absolute contract adherence;
5. the low-cost model improves under G8/G9 or reaches the declared pass threshold;
6. the composition interface is declared;
7. cross-step carried obligations are explicit when a macro composes multiple atoms;
8. unsupported claims and non-claims are updated before expansion.

These criteria prevent a broad workflow result from entering the claim set before its mechanism and failure mode are visible. They are stricter than ordinary prompt evaluation for that reason.

3.6 Repair-Loop Protocol

The repair-loop protocol is:

1. observe a real model failure;

2. isolate the missing mechanism or obligation;
3. make the obligation explicit in the input and output contract;
4. add or update a known-bad fixture that captures the failure;
5. run local golden/bad regression;
6. execute a targeted real-model slice;
7. update the evidence ledger, claim boundary, and backlog before expanding scope.

Stage 7e provides the first complete example. Its first version found that the low-cost-model G9 run did not retain the stage gate or decision trace. Stage 7e v2 made those obligations explicit, but some outputs still omitted unknown state. Stage 7e v3 added an unknown-state requirement. The omission disappeared, although one output reduced known-state provenance to generic labels. Stage 7e v4 required explicit provenance and passed 4/4 targeted smoke runs after retry. Stage 7-next reused the repaired obligations in a method-plan update macro and passed 4/4 targeted smoke runs without provider errors.

Stage B extends the same loop to ablation and stability confirmation. The Stage B v5 protocol failed 0/4 strict runs because it did not preserve exact evidence arrays and because one exact gate field was absent from the model-visible contract. Stage B v5.1 exposed the complete gate, separated immutable evidence bindings, and passed 4/4. The next ablation isolated the evidence-binding representation but did not observe the preregistered large independent effect. Stage B v5.3 restored explicit state-removal operations through a structured transition delta and passed 15/15 strict runs, although the paired causal threshold was still not met. The protocol was then frozen before the Stage B v5.4 execution, which passed 40/40 fresh strict runs across five perturbations. These stages answer different questions: whether a bundled repair works, whether one component has a large independent effect, and whether the repaired protocol remains stable under repetition.

3.7 Metrics And Claim Rules

Each run emits `task_success`, `schema_validity`, `citation_grounding`, `state_accuracy`, `evidence_type_accuracy`, `stage_completion`, `trace_completeness`, `context_relevance`, and `atom_primary_metric`. Controlled-transition runs also report exact evidence-array preservation, residual-state accuracy, state-transition accuracy, complete-gate accuracy, retention-attestation accuracy, and a strict aggregate controlled-mutation decision.

Table 2 gives a compact reading guide for the core metrics. Detailed thresholds, fixture-specific checks, and evaluator outputs are provided in the reproducibility package.

Metric	What it checks	Evaluator type
<code>schema_validity</code>	Required fields and types are present	deterministic schema/field check
<code>citation_grounding</code>	Claims carry admissible evidence IDs	deterministic evidence-ID check
<code>state_accuracy</code>	Known and unknown state are preserved	fixture-specific state check
<code>evidence_type_accuracy</code>	Evidence type labels remain correct	evidence-type check
<code>stage_completion</code>	Required stage gates are preserved	stage-gate check
<code>trace_completeness</code>	Required decision and rejection traces are present	structured trace check

Metric	What it checks	Evaluator type
<code>context_relevance</code>	Required or excluded context obligations are respected	context-obligation check
<code>atom_primary_metric</code>	The atom-specific dominant obligation is satisfied	atom evaluator
<code>task_success</code>	The declared task contract passes	composite evaluator

The metrics measure contract adherence. They do not measure open-ended output quality or human preference.

Gap compression is computed only for a nonzero G0 baseline gap. When both G0 baselines collapse, or when the low-cost model improves enough to reopen the absolute gap in the opposite direction, the result is reported as mixed, undefined, or negative. It is not converted into a compression claim.

Weak-model enablement is reported when the low-cost model reaches a pass threshold under a harness condition after failing or underperforming under weaker conditions.

Only the 40 fresh v5.4 runs contribute to the Stage B stability rate; the 15-run v5.3 explicit-delta pilot is not pooled into that estimate. Pooled and per-condition rates use two-sided 95% Wilson intervals. The v5.3 comparison forms 15 matched pairs by perturbation condition and repetition and reports the absolute risk difference, discordant-pair counts, and a two-sided exact McNemar result. Fisher’s exact test was named in the original preregistration, but it treats the arms as independent. Its value remains in the audit record and is not the primary paired analysis.

The Stage B v5.3 experiment plan defined an engineering-effect threshold of 0.20 absolute risk difference. With 15 runs per arm, the treatment needed at least three additional passes. This coarse engineering decision gate was fixed before execution. It was not a conventional significance threshold, an equivalence margin, or a power-derived minimum detectable effect. The preregistration also contained no separate utility analysis for the cutoff.

4 Results

4.1 Overview

The original hypothesis holds only within a bounded scope. Contract-rich harnessing raises absolute contract adherence across several productivity-task settings. In highly constrained tasks with nonzero baseline gaps, it can also compress cross-model gaps. The recurring result, however, is weak-model enablement on bounded, contract-critical operations. When obligations are explicit and evaluated deterministically, the low-cost model can reach the declared pass level on tasks that were unstable under weaker harnessing or broader prompts.

Table 3 summarizes the main claim boundary.

Claim	Evidence	Boundary
Absolute contract adherence lift	broad slices, mechanism atoms, admitted macros	tested conditions only
Gap compression	all measured nonzero gaps compressed in structured extraction	conditional on nonzero baseline gaps and constrained tasks

Claim	Evidence	Boundary
Weak-model enablement	Stage 6, Stage 7r.1, Stage 7e v4, Stage 7-next, Stage B v5.4	bounded contract-critical operations
Controlled-transition stability	Stage B v5.4	one frozen protocol, model, provider, and perturbation suite
Full workflow reliability	not supported	remains a non-claim
Production readiness	not supported	remains a non-claim

Table 4 summarizes the main empirical layers and the claim each layer permits.

Layer	Stage	Runs	Failure -> Repair	Outcome	Allowed claim
Task slice	structured extraction	24	schema/tool gaps -> G9 packet	key gaps to 0	conditional gap compression
Task slice	project init	12	mixed metric movement -> G9	mixed	no universal compression
Task slice	research workflow	12	zero or mixed baseline gaps -> G9	mixed/undefined	absolute adherence only
Atom	Stage 6	48	mechanism failures -> atom contracts	low-cost lift	weak-model enablement
Macro	Stage 7p/v2	12	stale-context loss -> carried obligations	repaired in v2	composition retention
Macro	Stage 7e	6 + repairs	trace and state gaps -> explicit obligations	targeted smoke passed	fixed macro only
Macro	Stage 7-next	4	neighboring transfer -> reused obligations	targeted smoke passed	narrow transfer
Ablation	Stage B v5.2-v5.3	30 + 30	bundled repair -> isolated controls	no preregistered large independent effect	component-effect boundary
Stability	Stage B v5.4	40	passing repair -> frozen protocol	40/40 fresh passes	one frozen transition protocol

4.2 Task Slices: Absolute Lift, Conditional Gap Compression

The structured-extraction v2 slice is the only task slice in which every measured nonzero gap compressed to 0.000 under G9. Under G0, nonzero baseline gaps appeared on task success, schema validity, tool-call correctness, human acceptance, cost efficiency, and safety consistency. The resulting compression ratio was 1.000 on each of those metrics. Citation grounding had a baseline gap of 0.000 and is therefore reported as n/a rather than as compression.

The project-initialization slice shows why gap compression cannot be the universal claim. G9 compressed the task-success gap from 0.111 to 0.000 and the safety-consistency gap from 0.200 to 0.000. Schema validity moved in the opposite direction, from a baseline gap of 0.250 to an arm gap of 0.583, yielding a negative compression ratio of -1.333. Human acceptance and cost efficiency also showed negative compression ratios.

The research-workflow slice further weakens a universal gap-compression story. Several G0 baseline gaps were already 0.000, making compression undefined. G9 compressed schema-validity

gap from 0.067 to 0.000, but task-success gap became 0.083 from a 0.000 baseline and human-acceptance/cost-efficiency gap movement was slightly negative.

Taken together, the task slices show absolute lift and conditional gap compression. They do not show universal gap closure.

4.3 Mechanism-Atom Pilot: Broad Workflows Need Smaller Units

The Mechanism-Atom Pilot (Stage 6) completed 48/48 real-model runs after documented timeout recovery. The main result was weak-model enablement. Relative to its G0 baseline, the low-cost model under G9 gained +0.576 on task_success and +0.833 on both schema_validity and atom_primary_metric.

On the contract-critical metrics, low-cost model + G9 also scored above strong_model + G0. The differences were +0.743 for task_success and +1.000 for both schema_validity and atom_primary_metric.

General contract metrics showed mostly positive gap compression: G9 compression was 1.000 for task_success and 1.000 for schema_validity. The atom_primary_metric result remained mixed, with 0.000 compression under G9 and negative values under G2/G8. Thus, the low-cost model can improve on bounded operations even when atom-specific gap compression is not uniform.

4.4 Partial Macro Composition: Atom Success Is Not Enough

The Partial Macro Composition study (Stage 7p) tested whether passing atoms could compose into a narrow partial macro:

A10 bounded context recall -> A9 no-overwrite action planning -> A6 validator repair

All 6/6 real SiliconFlow runs completed. Strong_model G8 and G9 passed the full partial-composition chain. Under G8/G9, the low-cost model reached task_success=0.800 and 0.900, respectively, with schema_validity=1.000 and safety=1.000. It still failed the full chain because context_relevance remained 0.000: the composed output did not carry the stale-context exclusion forward explicitly.

Stage 7p v2 added an explicit composition-retention contract. The same partial chain then passed for both model tiers under G8/G9. For this macro, negative context constraints survived multiple atom outputs only when cross-step retention was explicit.

4.5 Atom Revision And Targeted Repair

The Atom Revision and Repair sequence (Stage 7r / 7r.1) began by redesigning six boundary-prone atoms: A2R, A3R, A4R, A5R, A7R, and A8R. Local gates passed: 6/6 fixture structures, 12/12 local golden/bad expectations, 36/36 packet compilation, and preflight with 0 errors and 0 warnings.

The real-model smoke completed 35/36 outputs. The single missing output was A8R low-cost G8, which repeatedly timed out under SiliconFlow and was treated as an execution deviation rather than a model-quality score. On completed runs, strong-model G8/G9 passed 12/12, while the low-cost model still failed strict A2R citation grounding and A7R trace completeness.

Stage 7r.1 targeted exactly those failures by tightening the contracts. A2R1 required every grounded claim to be an object with non-empty evidence_ids. A7R1 required rejected-option objects with evidence IDs and trace steps for C2 support, C1 rejection, and C3 rejection. The targeted 8-run low-cost-model smoke passed 8/8.

In these targeted atoms, narrowing the output contract repaired the low-cost-model failures in claim-level evidence binding and rejection-trace completeness.

4.6 Evidence-Decision Macro Repair (Stage 7e)

The Evidence-Decision Macro Repair sequence (Stage 7e) combined state inventory, evidence grounding, evidence-type separation, traceable decision, and stage-gated synthesis in one narrow evidence-bound macro. The first smoke completed 6/6 runs. Strong_model G8/G9 and low-cost-model G8 passed with `task_success=1.000` and `atom_primary_metric=1.000`. Both model tiers failed under G0. Low-cost-model G9 reached `task_success=0.714` but did not retain the complete decision trace and stage gate.

Stage 7e v2 made retention of `decision_trace`, `stage_gate`, and `carried_obligations` explicit. The targeted low-cost-model G8/G9 smoke completed 4/4 runs; `trace_completeness` and `stage_completion` were 1.000 in every run. Only 1/4 passed the full macro because the other outputs omitted unknown Git branch, CI status, or network/API approval state from `state_inventory`.

Stage 7e v3 addressed unknown-state retention. All four targeted runs preserved the required Git/CI/network unknown-state fields and forbidden-inference fields, and the full strict pass count rose to 3/4. The remaining G8 failure compressed known-state provenance into generic labels.

Stage 7e v4 required each `state_inventory.known_state[]` entry to contain `state_id`, `fact`, and `evidence_ids`. One provider timeout and one truncated output were retried. After those retries, all four targeted runs passed with `task_success=1.000` and `atom_primary_metric=1.000`. State accuracy, citation grounding, evidence type accuracy, trace completeness, and stage completion were also 1.000.

The sequence shows the repair loop directly. It does not show that the low-cost model became generally stronger. The harness identified the missing obligations, made them explicit, and checked them under a fixed macro contract.

4.7 Neighboring Macro Transfer (Stage 7-next)

The Neighboring Macro Transfer study (Stage 7-next) tested the same obligation set on an evidence-bound method-plan update rather than the original fixture. The macro reused the Stage 7e v4 obligations and added one stressor. Its output contract required the model to identify the next admitted macro, specify its admission criteria, preserve the local and real-model gates, and declare non-claims.

The local gate met 2/2 expectations. The golden output passed with `task_success=1.000` and `atom_primary_metric=1.000`. The known-bad output, which expanded prematurely to a broader workflow, failed with `task_success=0.000`.

The real smoke used the low-cost model only:

`Qwen/Qwen3-8B x G8/G9 x 2 repetitions = 4 runs`

All four targeted runs completed without provider errors, timeouts, or truncated-output retries. Every reported metric was 1.000: `task_success`, `atom_primary_metric`, `schema_validity`, `citation_grounding`, `state_accuracy`, `evidence_type_accuracy`, `trace_completeness`, `stage_completion`, and `context_relevance`.

Within this scope, the result supports transfer of the Stage 7e v4 obligations to one closely related fixed method-plan macro with one new explicit stressor.

4.8 Controlled State-Mutation Study (Stage B)

The Controlled State-Mutation Study (Stage B) does not introduce a general state-transition method. It separates three parts of the repair-loop evidence: bundled repair, component effect, and stability of a frozen protocol.

The task contained one controlled mutation: move network API approval from unknown to known. The output had to preserve the exact evidence bindings, residual unknown state, and residual forbidden inferences. It also had to record the transition, complete gate, and retention attestation.

The first state-transition smoke, Stage B v5, passed 0/4 under the strict aggregate. All four runs preserved the schema, residual state, transition, and attestation, but they did not preserve the exact evidence arrays. The model-visible gate also omitted the exact expected `next_action` value. Stage B v5.1 exposed the complete gate and separated immutable evidence bindings from editable prose. That protocol passed 4/4, although the revision bundled two changes and the run count was too small for a stability estimate.

Stage B v5.2 isolated evidence-binding representation in a preregistered 30-run ablation. The binding-separated arm passed 15/15 exact-array checks; the claim-coupled arm passed 14/15. Their risk difference was 0.067, below the 0.20 engineering threshold. Each arm passed only 10/15 strict aggregates because nine runs retained an obsolete forbidden-inference entry. The ablation therefore did not support a large independent effect from evidence-binding separation.

Stage B v5.3 addressed that state error. Both arms used the same initial state, exact final postconditions, evidence bindings, event, gate, attestation, perturbations, and evaluator. The only treatment addition was a structured `required_transition_delta` naming the values to remove, add, and preserve. Across 30 fresh runs, the explicit-delta arm passed 15/15 strict and residual-state checks; the exact-postcondition-only arm passed 13/15. Both arms passed 15/15 on evidence, transition, gate, schema, and attestation.

The 30 runs form 15 matched pairs by perturbation condition and repetition. Thirteen pairs were pass/pass. Two paired an explicit-delta pass with a postcondition-only failure, and none favored the postcondition-only arm. The two-sided exact McNemar result was $p=0.500$. The residual-state risk difference was 0.133, below the preregistered 0.20 threshold. The causal result is mixed: the treatment passed all 15 runs, but the experiment did not establish the planned engineering-scale independent effect over an already strong baseline.

Stage B v5.4 used a separate preregistered question: would the frozen explicit-delta protocol maintain high absolute adherence over 40 fresh executions? It reused the five frozen v5.3 treatment fixtures without changing the prompts, evaluator, thresholds, provider settings, or output contract. The five conditions were canonical, field alias, evidence order shuffled, distractor evidence, and unknown-state paraphrase, each repeated eight times.

All 40 runs passed both the strict controlled-mutation metric and every component metric. Each perturbation condition passed 8/8. The pooled strict rate was 1.000, with a two-sided 95% Wilson interval of [0.912, 1.000]. For each individual 8/8 condition, the interval was [0.676, 1.000].

Condition	Passes	Rate	95% Wilson interval
canonical	8/8	1.000	[0.676, 1.000]
field alias	8/8	1.000	[0.676, 1.000]
evidence order	8/8	1.000	[0.676, 1.000]
distractor evidence	8/8	1.000	[0.676, 1.000]
unknown-state paraphrase	8/8	1.000	[0.676, 1.000]

Condition	Passes	Rate	95% Wilson interval
pooled	40/40	1.000	[0.912, 1.000]

All 40 calls returned valid JSON. There were zero provider errors and zero retries. Median latency was 19.500 seconds, P90 latency was 22.183 seconds, and usage totaled 83,312 prompt tokens and 19,672 completion tokens.

The supported stability statement is:

Under the frozen explicit-transition-delta G9 protocol, Qwen3-8B completed the tested controlled multi-array state mutation in 40/40 fresh runs across five perturbation conditions.

This stability result does not change the mixed causal result from the v5.3 ablation. It also does not establish arbitrary state-machine reliability, tool execution, rollback, concurrency, task-family generalization, or production readiness.

5 Discussion And Limitations

5.1 What The Results Mean

Some reliability requirements can be stated outside the model as explicit contracts. In these experiments, low-cost models completed bounded tasks that had been unstable under weaker or broader prompts once task state, admissible evidence, and output shape became inspectable. Stage gates, trace requirements, and carried obligations supplied the corresponding workflow controls.

Several failures are more specific when read this way. The decision trace appeared only after it became structurally required. Enumerating unknown state removed the corresponding omissions, while provenance-bearing state objects addressed the later provenance compression. Partial macro composition passed only after cross-step obligations were explicit. Each failure identifies an obligation that can be named and tested rather than only a lower-quality answer.

Stage B qualifies the interpretation. An explicit obligation can be part of a stable passing protocol without showing a large independent causal effect against every strong alternative specification. The v5.3 baseline already exposed exact final postconditions and passed 13/15. The explicit delta removed the two observed failures, but the result did not reach the preregistered effect threshold. With only two discordant pairs, the paired comparison has little power for small effects. It establishes neither equivalence nor the absence of a modest benefit. Stage B v5.4 therefore provides bounded absolute-stability evidence, not proof that an explicit delta is universally necessary or sufficient.

5.2 Gap Compression Is Conditional

Model capability gap compression motivated the original study, and the structured settings provide some support for it. In structured extraction, every measured nonzero gap compressed to 0.000 under G9; the input, output, and correctness criteria were also tightly constrained. Project initialization and research workflow were less consistent: absolute contract adherence improved, while gap movement was mixed, undefined, or negative depending on the metric.

Gap compression is therefore an outcome that depends on the task, metric, baseline gap, and harness arm. It is not the general thesis. The more stable evaluation question is whether explicit harnessing lets the low-cost model reach a declared contract-adherence threshold.

5.3 Why Negative Results Matter

The negative and partial results identify where the method changed. Stage 7p v1 showed that passing atoms do not automatically compose. Stage 7r improved low-cost-model performance without repairing every contract-critical behavior. Stage 7e v2 and v3 exposed the missing obligations later addressed in Stage 7e v4.

Removing those results would hide the repair path on which the final protocol depends.

5.4 Bounded Macros Are Not Full Workflows

Stage 7e v4 and Stage 7-next use fixed inputs, no tools, and deterministic evaluation. They do not include live source discovery, file mutation, external tool execution, or changing workspace state. The interpretation must stay within that boundary.

The supported statement is:

Low-cost models can complete bounded evidence-bound macros when reliability obligations are explicit.

It should not be stated as:

Low-cost models can reliably run full project initialization or full research workflows.

Full workflows add live execution problems such as tool selection, permission handling, and filesystem mutation. They must also handle source volatility and partial failures while maintaining long-horizon memory, user clarification, and multi-step state updates. The method offers a way to approach these problems, but the current experiments do not show that the workflows are solved.

5.5 Deterministic Evaluation, Sample Size, And Runtime Effects

The evaluation pipeline uses deterministic evaluators, golden outputs, and known-bad outputs. Pass/fail decisions can therefore be audited and repeated without relying on subjective preference scores. The scope is correspondingly narrow: the metrics cover contract adherence, not prose quality, human usefulness, creative insight, or open-ended judgment.

The evaluator can overfit to its known failures. A known-bad suite contains only failure modes that were anticipated or observed. Stage B v5.3-v5.4 adds field-name, evidence-order, unknown-state-language, and distractor-evidence perturbations, all of which passed. These tests still form a designed five-condition suite. The experiments do not cover arbitrary schema or event-order changes, adversarial evidence, or multiple transitions. Rollback, concurrent updates, and live tool state also remain untested.

Several repair experiments are targeted smoke tests with small run counts. Stage 7e v4 used four runs after retry, and Stage 7-next used four runs. Stage B v5.4 adds repetition for one controlled-transition protocol, not task diversity. Each perturbation condition contains eight runs, with an 8/8 Wilson interval of [0.676, 1.000]. The pooled interval describes repeated success within the frozen fixture family; it is not a population estimate over agent tasks.

Provider behavior affected earlier experiments. Some SiliconFlow runs timed out. Stage 7e v4 required one retry after a timeout and another after a truncated output. Stage 7-next, Stage B v5.3, and Stage B v5.4 completed without provider errors or retries. Because the study uses one provider and cannot control provider-side batching, hardware, or service changes, runtime deviations remain a validity threat.

All provider-backed evidence comes from SiliconFlow. A provider-independent reliability claim would require replication across providers and model families, so no such claim is made here.

5.6 PEtFiSh Specificity And Harness Cost

The fixtures and workflows come from the PEtFiSh project. Its skills, packs, evidence ledgers, and backlog structures may not generalize to other agent systems. PEtFiSh is therefore the implementation context. The transferable contract objects are task specifications, bounded memory, evidence bundles, and output contracts. Workflow gates, trace logs, validators, known-bad cases, and repair loops provide the enforcement and repair process around those objects.

The claim concerns the contract stack and repair-loop protocol, not a particular pack catalog, skill name, or project directory convention.

Contract-driven harnessing adds engineering overhead through fixture design, schemas, evidence bundles, evaluators, and local gates. Manifests, event logs, and postprocessing add further execution and audit cost. Stage B v5.4 used 102,984 total tokens and had a 19.500-second median latency across 40 G9 runs. No matched G0/G9 and weak/strong-model overhead matrix is available. The evidence therefore cannot show that a harnessed low-cost model is cheaper per successful task than direct use of a stronger model. On simple tasks, the overhead may exceed the benefit. Strict contracts may also stabilize a strong model while reducing its flexibility.

5.7 Future Work

The next experiment must choose between economics and breadth. An economics study can compare Qwen3-8B and DeepSeek-V3.2 under G0 and G9 on the same frozen macro, using cost per successful contract pass, token use, latency, and retries as primary outcomes. A breadth study can add multiple transition events, rollback, event ordering, live tool state, or a second task family. Either study requires a new preregistration. The current 40-run result does not answer those questions.

6 Conclusion

Contract-driven harness engineering places some reliability obligations in explicit contracts, where failures can be observed, repaired, and covered by regression tests. Model quality still matters. The bounded result is that part of agent reliability can be engineered outside the model for some productivity tasks.

The evidence supports weak-model enablement on bounded, contract-critical operations. Gap compression is conditional: it is clearest in structured extraction, appears on some contract metrics in mechanism and partial-macro tests, and remains mixed or undefined in the broader project-initialization and research-workflow slices.

The repair loop begins by turning a failure into a named obligation and a contract revision. A known-bad fixture and local regression gate then preserve the failure case, while targeted ablation and fresh stability testing determine what the revision supports. The claim boundary is updated last. For the controlled transition, this process ended with 40/40 strict passes across five perturbations. The paired ablation did not meet its preregistered large-effect threshold. The repaired protocol was stable in the tested setting, while its independent causal contribution and broader generality remain unresolved.

7 Appendix A. Current Non-Claims

This paper should not claim:

- low-cost models are generally equivalent to strong models;
- harnessing universally compresses model gaps;
- full project initialization is solved;
- full research workflow is solved;
- the harness is production ready;
- fixed macro success implies open-ended tool-using workflow reliability;
- explicit transition delta has a proven 0.20 causal advantage over exact postconditions;
- one 40-run fixture family establishes arbitrary state-machine reliability;
- current evidence establishes a favorable cost or latency tradeoff against a strong model.

8 Appendix B. Contribution-To-Evaluation Alignment

Contribution	Evaluation support	Boundary
Contract-driven harness model	Task slices and methods artifacts	Method definition, not a production-readiness claim.
Mechanism atoms	Atom definition, coverage framework, Stage 6-7 atom results	Atom pass does not prove workflow pass.
Conditional gap compression	Structured extraction, project initialization, research workflow slices	Compression only when baseline gaps are nonzero and gap movement is not reversed.
Repair-loop protocol	Stage 7e v1-v4 and Stage B v5-v5.4	Fixed evidence-decision and controlled-transition macros.
Bounded weak-model enablement	Stage 7e v4, Stage 7-next, and Stage B v5.4	Fixed-input, no-tool, deterministic macros.
Controlled-transition stability	Stage B v5.4	One frozen model/provider/harness/fixture family.
Independent explicit-delta effect	Stage B v5.3	Mixed; preregistered 0.20 threshold not met.

9 Appendix C. Evidence Traceability Matrix

Paper claim	Evidence IDs	Source IDs	Status
Contract-rich harnessing improves absolute contract adherence and can compress gaps under constrained conditions.	P2-E28, P2-E30, P2-E32, P2-E33	P2-SILICONFLOW-V2-FULL24, P2-SILICONFLOW-PROJECT-INIT-12, P2-SILICONFLOW-RESEARCH-WORKFLOW-12, P2-CLAIM-BOUNDARY-MEMO	Supported with conditional wording.
Structured extraction is the task slice in which every measured nonzero gap compressed to 0.000 under G9.	P2-E27, P2-E28	P2-SILICONFLOW-V2-FULL24	Supported for tested SiliconFlow v2 slice.
Project initialization and research workflow do not support universal gap compression.	P2-E30, P2-E32, P2-E33	P2-SILICONFLOW-PROJECT-INIT-12, P2-SILICONFLOW-RESEARCH-WORKFLOW-12, P2-CLAIM-BOUNDARY-MEMO	Supported; use mixed/undefined wording.
Mechanism atoms make broad workflow failures interpretable.	P2-E35, P2-E36, P2-E56, P2-E60	P2-MECHANISM-ATOM-DEFINITION, P2-MECHANISM-ATOM-COVERAGE, P2-STAGE7R-REVISED-ATOMS, P2-STAGE7R1-A2R-A7R-SMOKE	Supported as methodology and targeted empirical repair evidence.
Atom success does not automatically imply macro composition success.	P2-E51, P2-E52	P2-STAGE7P-PARTIAL-COMPOSITION	Supported by Stage 7p v1 failure.
Explicit cross-step carried obligations can repair the Stage 7p composition failure.	P2-E53, P2-E54	P2-STAGE7P-V2-COMPOSITION-RETENTION	Supported for A10 -> A9 -> A6 partial macro.
Stage 7r.1 repaired low-cost-model A2R/A7R failures through tighter output contracts.	P2-E57, P2-E58, P2-E59, P2-E60	P2-STAGE7R1-A2R-A7R-PREP, P2-STAGE7R1-A2R-A7R-SMOKE	Supported for targeted atoms only.

Paper claim	Evidence IDs	Source IDs	Status
Stage 7e v1-v4 demonstrates a repair-loop protocol for a fixed evidence-decision macro.	P2-E62, P2-E64, P2-E66, P2-E68, P2-E69, P2-E70	P2-STAGE7E-EVIDENCE-DECISION, P2-STAGE7E-V2-RETENTION, P2-STAGE7E-V3-STATE-RETENTION, P2-STAGE7E-V4-KNOWN-STATE-PROVENANCE, P2-CLAIM-BOUNDARY-MEMO	Supported with fixed-input/no-tool boundary.
Stage 7-next supports transfer of Stage 7e v4 obligations to one neighboring method-plan macro.	P2-E72, P2-E74, P2-E75	P2-STAGE7-NEXT-METHOD-PLAN-LOCAL, P2-STAGE7-NEXT-METHOD-PLAN-SMOKE	Supported as narrow transfer evidence.
Evidence-binding separation did not show the preregistered large independent effect.	P2-E160, P2-E161, P2-E162, P2-E163, P2-E164	P2-STAGE-B-V52-EVIDENCE-BINDING-ABLATION-LOCAL, P2-STAGE-B-V52-EVIDENCE-BINDING-ABLATION-EXECUTION, P2-STAGE-B-V52-EVIDENCE-BINDING-ABLATION-EVALUATION, P2-STAGE-B-V52-EVIDENCE-BINDING-ABLATION-FAILURE-AUDIT, P2-STAGE-B-V52-EVIDENCE-BINDING-ABLATION-DECISION	Supported as a bounded negative ablation result.
Explicit transition delta passed 15/15, but the 0.133 risk difference missed the preregistered 0.20 threshold; exact McNemar $p=0.500$.	P2-E165, P2-E166, P2-E167, P2-E168, P2-E176	P2-STAGE-B-V53-EXPLICIT-DELTA-ABLATION, P2-STAGE-B-V53-PAIRED-ANALYSIS-CORRECTION	Supported as a mixed causal result.

Paper claim	Evidence IDs	Source IDs	Status
The frozen explicit-delta protocol passed 40/40 fresh runs across five perturbation conditions.	P2-E169, P2-E170, P2-E171, P2-E172	P2-STAGE-B-V54-EXPLICIT-DELTA-STABILITY	Supported as bounded absolute stability, not task-family or state-machine generality.
Full project initialization, full research workflow, production readiness, and general model equivalence remain non-claims.	P2-E33, P2-E63, P2-E69, P2-E70, P2-E75	P2-CLAIM-BOUNDARY-MEMO, P2-STAGE7E-EVIDENCE-DECISION, P2-STAGE7E-V4-KNOWN-STATE-PROVENANCE, P2-STAGE7-NEXT-METHOD-PLAN-SMOKE	Supported as explicit boundary.
Related work: orchestration, declarative programs, structured outputs, retrieval/tools, memory, verification, and skill ecosystems are adjacent lines.	P2-E05, P2-E06, P2-E07, P2-E08, P2-E09, P2-E83, P2-E84, P2-E85, P2-E86, P2-E87, P2-E88, P2-E89, P2-E90, P2-E91, P2-E92, P2-E93, P2-E94, P2-E95, P2-E96, P2-E98	External source IDs listed in <code>source-index.md</code>	Supported as background; convert to publication-style citations before submission.

10 Reproducibility Package

The project repository provides a public reproducibility package, with additional local artifacts under `research/` [4]. It contains the source index, evidence ledger, mechanism-atom definitions, macro fixtures, prompt manifests, provider event logs, model-output artifacts, deterministic evaluator outputs, metric summaries, stage reports, citation audit reports, and citation metadata.

Repository: <https://github.com/kylecui/contract-driven-harness-study>.

Claims can be audited in four steps:

1. Read Appendix C to map each paper claim to evidence IDs.
2. Open `research/03_evidence/evidence-ledger.jsonl` and locate those evidence IDs.
3. Inspect the referenced stage report, metric summary, fixture, validator output, provider event log, and model-output artifact.
4. Re-run the deterministic local gate for the corresponding fixture when a fixture/evaluator pair is provided.

For example, the Stage B v5.4 claim maps to P2-E169 through P2-E172. The corresponding audit includes the preregistration, freeze manifest, prompt manifest, 40 execution records, raw outputs, deterministic evaluation, analysis, and freeze-integrity audit.

Where available, stage reports record the command or script path used to regenerate local evaluator outputs. The repository README and method scripts are the entry points for rerunning local gates and inspecting artifacts.

The core traceability files are:

- `research/01_sources/source-index.md`
- `research/01_sources/contract-driven-harness-citation-metadata.md`
- `research/03_evidence/evidence-ledger.jsonl`
- `research/06_outputs/contract-driven-harness-compact-results-appendix.md`
- `research/07_reviews/contract-driven-harness-citation-audit.md`
- `research/07_reviews/contract-driven-harness-source-coverage.md`
- `research/07_reviews/contract-driven-harness-unsupported-claims.md`

External references are prepared in `research/06_outputs/contract-driven-harness-references.bib`. Local empirical claims should be checked against Appendix C and the evidence ledger rather than treated as ordinary literature citations.

11 Bibliography

The BibTeX bibliography for this working draft is maintained in `research/06_outputs/contract-driven-harness-references.bib`.

For arXiv preparation, compile this manuscript with that BibTeX file and keep Appendix C as the evidence traceability layer. For ACM or IEEE submission, move most local evidence IDs to supplementary material and cite the local artifact bundle as a reproducibility package.

References

- [1] Soufiane Amini et al. Open agent specification (agent spec): A unified representation for ai agents, 2025. arXiv v4 as of 2025-11-07.
- [2] Anthropic. Building effective agents. <https://www.anthropic.com/engineering/building-effective-agents>. Accessed 2026-06-09.
- [3] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure ai agents, 2025. arXiv v1 as of 2025-05-06.
- [4] Contract-Driven Harness Study. Local experiment artifacts and evidence ledger. Local reproducibility package under `research/`, 2026. Includes source index, evidence ledger, stage reports, metrics files, event logs, macro fixtures, and paper drafts.
- [5] dottxt-ai. Outlines documentation. <https://dottxt-ai.github.io/outlines/latest/>. Accessed 2026-06-09.

- [6] Guardrails AI. Validators documentation. <https://guardrailsai.com/docs/concepts/validators/>. Accessed 2026-06-09.
- [7] Omar Khattab et al. DSPy: Compiling declarative language model calls into self-improving pipelines, 2023.
- [8] LangChain. Langgraph documentation. <https://docs.langchain.com/oss/python/langgraph>. Accessed 2026-06-09.
- [9] Alexander W. Lee, Justin Chan, Michael Fu, Nicolas Kim, Akshay Mehta, Deepti Raghavan, and Ugur Cetintemel. Semantic integrity constraints: Declarative guardrails for ai-augmented data processing systems, 2025. arXiv v3 as of 2025-08-07; also PVLDB 18(11):4073–4080, 2025.
- [10] Letta. Agents overview documentation. <https://docs.letta.com/guides/agents/overview/>. Accessed 2026-06-09.
- [11] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks, 2020.
- [12] Microsoft. Autogen documentation. <https://microsoft.github.io/autogen/>. Accessed 2026-06-09.
- [13] Microsoft. Semantic kernel documentation. <https://learn.microsoft.com/en-us/semantic-kernel/>. Accessed 2026-06-09.
- [14] OpenAI. Structured outputs guide. <https://platform.openai.com/docs/guides/structured-outputs>. Accessed 2026-06-09.
- [15] Charles Packer et al. MemGPT: Towards LLMs as operating systems, 2023.
- [16] Shishir G. Patil et al. Gorilla: Large language model connected with massive APIs, 2023.
- [17] Timo Schick et al. Toolformer: Language models can teach themselves to use tools, 2023.
- [18] SiliconFlow. Chat completions and model documentation. <https://docs.siliconflow.cn/cn/api-reference/chat-completions/chat-completions>. Accessed 2026-06-09; model list also consulted at <https://docs.siliconflow.cn/cn/userguide/models>.
- [19] Pengcheng Wang et al. AgentSPEX: An agent specification and execution language, 2026.
- [20] Melwin Xavier, Vaisakh M A, Melveena Jolly, and Midhun Xavier. Agentproof: Static verification of agent workflow graphs, 2026. Submitted 2026-03-20.
- [21] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models, 2022.
- [22] He Zhu, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao, Hanhao Li, Ningning Wang, Pai Liu, Tianhao Peng, Xin Gui, Xiaowan Li, Yuhui Liu, Yuchen Eleanor Jiang, Jun Wang, Changwang Zhang, Xiangru Tang, Ge Zhang, Jian Yang, Minghao Liu, Xitong Gao, Jiaheng Liu, and Wangchunshu Zhou. OAgents: An empirical study of building effective agents, 2025. arXiv v2 as of 2025-06-23.